

# **Evidence-Bearing Host Memory for LLM Assistants**

Anthony Paterson

Fractal Media Infrastructure

Gracemere, Queensland, Australia

fractalmediainfrastructure@gmail.com

Author manuscript preprint, v0.1

Not peer reviewed.

# Evidence-Bearing Host Memory for LLM Assistants

User-Controlled Recall, Exact Evidence, and Governable Continuity

## ABSTRACT

Long-term memory in large language model (LLM) systems is often described as if the model itself remembers. In practice, deployed assistant memory is usually a host-side architecture that records, summarizes, retrieves, injects, and revises context around a stateless or limited-context model. This paper argues that treating memory as host-owned infrastructure rather than model-owned recollection clarifies several design problems: context-window limits, summary drift, stale personalization, semantically adjacent but wrong retrieval, hidden state, and user uncertainty about why an assistant "knows" something. It proposes an evidence-bearing host-memory architecture that separates four roles: hot memory or active context, rolling summary, semantic routing or cold atlas, and cold evidence logs. Hot memory contains bounded material under current manipulation. Rolling summary orients the current task without preserving every detail. Semantic routing stores compact, reviewable references to historical evidence. Cold evidence logs preserve exact historical records, event order, source identity, and provenance. Recall is then treated as a deliberate host operation: search proposes, the user or policy selects, and the host injects exact bounded evidence into the next model turn under a visible boundary. An implementation-oriented case illustrates the architecture through append-oriented cold logs, deliberate recall, semantic placements, cold atlas records, bounded evidence hydration, one-turn attachment boundaries, and recall, attachment, feedback, sorting, and placement receipts. The resulting design does not provide infinite or human-like memory. It provides bounded, inspectable, user-controllable continuity while preserving the distinction between remembered shape and exact evidence.

## CCS CONCEPTS

Human-centered computing -- Interactive systems and tools; Human-centered computing -- Human computer interaction (HCI); Information systems -- Information retrieval; Software and its engineering -- Software system structures.

## KEYWORDS

large language models; assistant memory; human-AI interaction; evidence provenance; user control; retrieval-augmented generation

## 1 Introduction

AI assistants increasingly promise memory. They remember preferences, projects, personal facts, instructions, documents, prior decisions, and ongoing work. In everyday product language, this is often described as if the model remembers the user. That description is convenient, but technically and ethically imprecise. Most LLMs do not carry ordinary human-like memory across sessions. Continuity is usually created by host systems that store records outside the model and place selected context back into future prompts.

The distinction matters. If memory is treated as a property of the model, users and designers may overlook the machinery that determines what is stored, summarized, retrieved, trusted, forgotten, or injected. If memory is treated as host-owned infrastructure, those operations can be made inspectable and governable. This paper proposes an evidence-bearing host-memory architecture for LLM systems. The core claim is simple: summaries and semantic indexes should help locate memory, but exact evidence should remain separately recoverable. The host should own persistence and context injection. The model may propose, interpret, and use memory, but it should not directly own historical truth.

This approach is motivated by several recurring problems. Context windows are finite. Transcript logs become too large to use directly. Summaries can drift or flatten important distinctions. Vector retrieval can return material that is nearby in embedding space but wrong for the current question. Personalization can

become stale, intrusive, or overconfident. Cross-model memory can hide semantic translation losses. Most importantly, users can lose track of why a system is saying it remembers something.

The proposed architecture separates memory into layers: active context, rolling summary, semantic routing, and exact cold evidence. It treats recall as a bounded descent from broad shape to exact detail. It also treats user selection, host injection, and durable receipts as first-class parts of memory, not UI afterthoughts.

The paper is conceptual but implementation-informed. It draws on three anonymized project clusters: a model-defined persistent-memory design, a dual cold-memory/deep-recall design, and a current local-first assistant-memory implementation. The first proposes model-specific semantic buckets, deterministic hot context injection, context routing packets, and reducer-governed updates. The second proposes dual cold memory, deep recall, evidence-first abstraction, and assumption freezing. The current implementation realizes a narrower version through audit logs, user-triggered recall, semantic bucket placements, cold atlas records, evidence hydration, and governance receipts.

## 2 Method and Claim Boundary

This article is an architecture and methods paper. It does not report a large benchmark, product deployment, or controlled user study. Its evidence base consists of implementation artifacts, tests, design notes, and worked interaction patterns from a local-first assistant-memory stack. These artifacts are used to support the existence and coherence of the proposed architecture, not to claim broad effectiveness.

The necessary evidence for the stated claim is therefore modest but concrete: the paper must show that memory roles can be separated, that exact evidence can remain recoverable beneath summaries and semantic routing, that recall can be mediated by the host rather than delegated to model freeform output, and that user-visible receipts can make memory behavior inspectable. Existing implementation surfaces and tests are appropriate evidence for those architectural claims.

Empirical validation becomes necessary only for stronger claims: that users understand the interface, that the architecture reduces false recall in practice, that it improves task performance, or that it generalizes across products, models, corpora, and user populations. Those are important future questions, but they are not prerequisites for the present contribution.

## 3 Background: RAG, Agent Memory, and Context Management

Retrieval-augmented generation (RAG) combines parametric model knowledge with non-parametric external stores. Lewis et al. (2020) introduced RAG as a way to improve knowledge-intensive generation by retrieving external documents and conditioning generation on them. Later surveys describe increasingly modular RAG systems with query routing, retrieval, augmentation, reranking, and generation stages.

RAG is important because it makes memory-like behavior practical: a model can answer using information not contained in its weights or current prompt. But RAG alone does not settle memory governance. It says little about whether retrieved material is current, authoritative, user-approved, privacy-scoped, or exact enough for the task. It can also blur the distinction between a document that was found and a memory that should shape the assistant's behavior.

Agent-memory systems address related problems. Generative agents store observations, synthesize reflections, and retrieve memories to support believable simulated behavior (Park et al., 2023). Reflexion agents store verbal feedback from prior attempts in an episodic memory buffer to improve later task performance without weight updates (Shinn et al., 2023). MemGPT frames LLM context management through an operating-system analogy, using hierarchical memory and interrupts to move information between memory tiers (Packer et al., 2023).

These systems show that external memory can extend what LLM applications can do. They also show why design vocabulary matters. Memory can mean retrieved documents, episodic logs, summaries, reflections, user preferences, tool feedback, current task state, or simulation traces. A system that collapses these into one memory surface risks treating all retrieved context as equally current and authoritative.

The architecture proposed here shares the intuition that LLM applications need memory tiers. Its distinctive emphasis is evidentiary: compressed memory should point down to exact records, and exact records should

remain separable from interpretation. It also emphasizes host authority: the model can suggest memory actions, but durable mutation and prompt injection should be governed by deterministic host processes and user-visible controls.

The user-control emphasis also connects to broader human-AI interaction and explanation work. Human-AI guidelines emphasize making system status, uncertainty, scope, and correction paths visible to users (Amershi et al., 2019). Explanation research emphasizes that useful explanations are socially situated rather than merely technical displays (Miller, 2019). Explanatory debugging shows the value of systems that expose machine inferences in a form users can correct (Kulesza et al., 2015). Evidence-bearing memory applies that orientation to long-term assistant continuity: the user should be able to inspect why memory appeared, whether it rests on exact evidence, and how stale or misleading memory can be corrected.

## 4 Memory as Host Infrastructure

The phrase "the model remembers" hides at least six operations:

1. recording an event;
2. deciding whether it matters;
3. summarizing or indexing it;
4. retrieving it later;
5. deciding whether it is current and relevant;
6. injecting it into model context.

These operations need not, and often should not, be controlled by the LLM itself. The LLM may help propose summaries, classify events, or explain relevance. But the host should decide what is persisted, how it is versioned, how it is retrieved, and when it is placed into the model's working context.

This host-owned framing has several advantages. It makes memory inspectable: users can see what is stored and why. It makes memory correctable: stale or misleading records can be marked without rewriting history. It makes memory scoped: sensitive records can inherit privacy restrictions. It makes memory auditable: summaries can point to evidence, and injection events can leave receipts. It also reduces anthropomorphic confusion by locating continuity in the system architecture rather than in a hidden model self.

The guiding rule is:

```
The model may propose memory.
The host owns persistence.
Evidence determines what can honestly be claimed.
The user should be able to inspect and correct the record.
```

This rule does not prevent automation. It prevents silent authority transfer. A system may automatically log events, propose summaries, or rank retrieval candidates. But when memory changes the assistant's representation of the user or the task, that change should remain traceable to host rules, evidence, and user-controllable policy.

## 5 Four Memory Roles

The architecture distinguishes four roles: hot memory, rolling summary, semantic routing, and cold evidence. Hot memory is active context. It contains exact material currently under manipulation: the current user request, active constraints, selected source excerpts, retrieved historical details, current hypotheses, and task-specific instructions. It is small, visible, and aggressively bounded.

Rolling summary is current orientation. It answers what the session or task is about, what has already been established, what remains unresolved, and what should stay active. It is useful precisely because it is compressed, but it should not be treated as exact evidence.

Semantic routing is remembered shape. It includes semantic buckets, context routing packets, cold atlas records, and other compact structures that help the host find relevant history cheaply. These records can store

titles, summaries, currentness, confidence, tags, adjacency, and evidence references. They locate likely relevant material.

Cold evidence is historical specificity. It stores exact records: transcript blocks, event order, source paths, timestamps, tool outputs, source hashes, relation atoms, modification receipts, and provenance. It substantiates claims. It should be append-oriented and difficult to rewrite silently.

The relationship can be summarized:

```
Hot memory manipulates.
Rolling summary orients.
Semantic routing locates.
Cold evidence substantiates.
```

Figure 1 sketches the intended flow.

**Figure 1. Evidence-bearing host-memory flow.** A user request reaches a host memory governor. The governor can consult hot memory, rolling summary, semantic routing or cold atlas records, and cold evidence logs. Semantic routing points down to evidence rather than replacing it. Exact bounded evidence is hydrated from the cold log, previewed for user or policy selection, injected into the next model turn under a visible host boundary, and then cleared according to attachment policy. Recall, attachment, and feedback receipts return to the host as reviewable records of memory behavior.

The figure is deliberately asymmetrical. The model is not the center of persistence. The host mediates between user request, stored evidence, recall preview, and one-turn injection. The model can use the selected evidence once it is injected, but the historical record and memory mutation path remain outside the model's freeform completion stream.

The critical design point is that these roles must not collapse. A summary is not a transcript. A semantic bucket is not the source event. A retrieved candidate is not automatically current truth. A model's recollection-like output is not evidence unless it can descend to a stored record.

## 6 Dual Cold Memory and Deep Recall

One cold layer is not enough. Raw logs preserve exact detail but are too large and noisy for routine prompting. Summary-only memory is cheap but can flatten sequence, directionality, uncertainty, and contradiction. A dual cold-memory design uses both.

The cold atlas or semantic routing layer stores broad shape:

```
Several prior review failures involved evidence substitution.
Prior memory work distinguishes cold logs from current truth.
The user prefers local-first AI tooling with visible receipts.
```

The cold evidence layer stores exact detail:

```
event index;
timestamp;
source path;
speaker/source;
exact transcript block;
tool result;
bounded surrounding exchange;
receipt ID;
```

The four roles can be stated as a memory-lane table:

| Lane                          | Main function  | Typical content                                                                                    | Governance rule                                            |
|-------------------------------|----------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Hot memory                    | Manipulation   | Current request, active evidence excerpts, task constraints, selected recalled blocks              | Small, visible, and bounded to the present task            |
| Rolling summary               | Orientation    | Current task state, established decisions, unresolved questions, next actions                      | Useful as compressed context, but not exact evidence       |
| Semantic routing / cold atlas | Location       | Bucket placements, routing packets, broad historical shape, currentness, confidence, evidence refs | Locates candidate history; does not become truth by itself |
| Cold evidence log             | Substantiation | Transcript blocks, event order, source paths, tool outputs, hashes, receipts, provenance           | Append-oriented source of exact recall and audit           |

The central design question is not which lane is more intelligent. It is which lane is allowed to support which kind of claim.

Deep recall is the deliberate descent from shape to specificity. A broad memory may say that something relevant happened. Deep recall asks what exactly happened, in what order, and with what evidence.

This matters because many tasks require exactness only at the edge. A general conversation may need only orientation. A technical audit, claim correction, publication decision, or safety judgment may need exact wording, sequence, and provenance. The memory system should therefore support both cheap recall and deep recall.

Cheap recall:

```
query -> semantic/atlas match -> compact context
```

Deep recall:

```
query -> semantic/atlas match -> evidence refs -> exact hydration ->
bounded hot context
```

Honest failure is part of the design. If the system can find a broad memory but not exact evidence, it should say so. "I found the gist but cannot recover the source detail" is better than plausible reconstruction.

A worked recall example shows the difference:

User: What did we decide about memory attachments last time?

Summary-only answer:

We decided memory attachments should be temporary and user-controlled.

Evidence-bearing host-memory answer:

Broad match found in cold atlas: "recall attachment boundary."

Exact evidence hydrated from cold\_log.jsonl events 812-818.

Preview shown to user:

```
- selected cold context is injected under USER-SELECTED COLD LOG
CONTEXT
```

```
- attachment is cleared after prompt construction
```

```
- attachment receipt is written without copying transcript payload
```

User selects the evidence.

Model receives the bounded excerpt and answers:

```
We decided that selected cold-log context may shape the next response
only,
then must be cleared after prompt construction. The receipt records the
attachment metadata but does not replace or rewrite the cold log.
```

The second answer is not better because it is longer. It is better because it exposes the route from broad memory to exact evidence, gives the user a selection point, and preserves what remains unsupported.

## 7 Model-Native Buckets and Host-Governed State

A model-defined persistent-memory design adds a further idea: let a target model help define semantic buckets for a domain, then freeze those buckets as versioned host artifacts. The model defines the drawers; the host controls the notebook.

This is not a claim of model consciousness or introspection. "Model-native" means that the bucket map is generated by and versioned for a particular target model, prompt, source vocabulary, and bucket count. It recognizes that different models may carve a domain differently. Cross-model memory should therefore be migrated deliberately rather than treated as universal sameness.

A memory bucket might store user preferences, active projects, stable facts, pending actions, temporal context, or relationship context. The model may later propose reducer actions such as remember, append evidence, mark stale, request confirmation, or clear a bucket. But reducers, not the model's freeform text, determine what changes.

This structure addresses two opposing risks. A fully human-defined schema may not match how the target model best uses compact context. A fully model-owned memory may mutate by plausibility, drift, or hidden preference. Model-defined buckets plus host-governed reducers preserve both semantic usefulness and auditability.

The design also treats UNASSIGNED as a feature. Information that does not fit the current bucket map should not be discarded or forced into the nearest category. It should be stored as overflow or review material. Repeated overflow can indicate that the bucket map needs revision.

## 8 Deterministic Hot Context and Routing Packets

Standing memory gives broad continuity, but it should not force every specific fact into every prompt. Deterministic hot context injection addresses this by scanning user input for registered entities, aliases, or patterns, then injecting only relevant memory cards.

If the user mentions a known project, pet, person, device, module, or concept, the host can inject a compact card. The key is that the validator, not the LLM's vibes, selects the card. User text can contain matchable entities, but it cannot command the validator to load hidden memory.

Context routing packets extend this idea. A packet is not just a memory snippet; it is a navigation card. It says what the named thing is, its current role, status, adjacent concepts, evidence refs, and where to look next if deeper context is needed. For a large project ecosystem, packets turn mention into orientation.

Search returns "here is a place the term appeared." A routing packet returns "here is what this thing currently is, where it sits, and which nearby things may matter." This shifts token use from rediscovering context to reasoning with it.

Routing packets should remain compact and source-aware. They should not become giant summaries. They should carry currentness, confidence, and evidence links. Stale or archived packets should not masquerade as current project state.

## 9 Local Assistant Memory as an Implementation-Oriented Case

Current local assistant-memory work illustrates a narrow, practical version of the architecture.

The logging role records audit-history events to `memory/cold_log.jsonl`, provides search where embeddings are available, writes a rolling summary to `memory/lukewarm.txt`, and maintains module

status summaries. The user-facing chat layout also distinguishes Hot Memory or active context from Luke Warm or rolling summary.

The newer host-owned memory governor powers a deliberate recall path. It reads `memory/cold_log.jsonl` as exact historical evidence without rewriting it. It limits the first retrieval slice to user/assistant interaction events, extracts the matched exchange plus nearby bounded exchanges, preserves source path and event indexes, and renders selected cold context under a host-owned boundary.

The crucial rule is that recall is deliberate retrieval, not automatic belief. Search proposes candidates. The user selects. The host injects only selected exact transcript blocks into the next main assistant turn. Those attachments are cleared after prompt construction so cold evidence does not remain silently active across turns.

The system also records compact receipts. Recall sessions, attachments, and feedback are written to separate JSONL files linked by recall IDs. Feedback such as stale, misleading, or noisy does not delete cold logs. It becomes a quality signal for future retrieval and review.

The semantic memory lane adds another layer. The cold semantic index stores bucket maps, placements, sift proposals, and receipts over cold evidence references. It does not copy transcript text as replacement memory. Proposed placements remain inert until the user approves or rejects them. Approved placements are used for semantic recall lookup, after which exact bounded context is hydrated from cold logs before user selection.

The cold atlas lane stores broader records with summaries, status, currentness, confidence, and evidence refs. Atlas hits are not injected as truth. They are hydrated through the governor into exact cold-log transcript candidates. Atlas summaries explain why a candidate surfaced; exact payload comes from evidence.

This implementation pattern can be summarized:

```
cold_log.jsonl preserves exact chronology
semantic placements store refs and reasons
cold atlas stores broad historical shape
Recall hydrates exact bounded evidence
user selection gates injection
receipts preserve recall, attachment, and feedback
```

The design remains intentionally modest. It does not auto-promote cold evidence into active context. It does not let the logging role, summarizer, or main assistant silently rewrite memory. It treats semantic structure as navigation over evidence, not evidence replacement.

The implementation case is presented as a system surface with observable controls, not merely as repository background. The current project ecosystem provides several inspectable signals:

| System surface               | Implementation signal                                | Paper relevance                                                                                                          |
|------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Cold log                     | <code>memory/cold_log.jsonl</code> design references | Exact historical evidence is stored separately from active prompt context                                                |
| Rolling summary              | <code>memory/lukewarm.txt</code> design references   | Summaries orient current work but are not treated as exact historical proof                                              |
| Recall control               | user manual and memory-recall tests                  | Recall is a deliberate operator action with statuses such as found, partially found, conflicting memories, and not found |
| One-turn attachment boundary | architecture notes and cold-memory bridge design     | Selected evidence can be injected into the next prompt and then cleared, reducing silent carryover                       |

| System surface            | Implementation signal                                                                                                                               | Paper relevance                                                                                    |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Recall receipts           | <code>remember_recalls.jsonl</code> ,<br><code>remember_attachments.jsonl</code> ,<br>and <code>remember_feedback.jsonl</code><br>design references | Search, attachment, cancellation, and quality feedback become reviewable memory behavior           |
| Semantic placements       | semantic-index implementation and architecture documentation                                                                                        | Semantic buckets store evidence references and review status rather than replacing transcript text |
| Cold atlas                | cold-atlas implementation and architecture documentation                                                                                            | Broad historical shape points down to exact evidence refs before recall                            |
| Feedback without mutation | tests and architecture notes for stale, misleading, or noisy feedback                                                                               | Memory quality signals do not silently delete or rewrite cold evidence                             |

This makes the paper empirically tractable. The implementation can be evaluated at the level of user-visible behavior and stored receipts: whether a query returns the expected status; whether selected evidence is attached only for the next turn; whether conflicting memories remain visible as conflicts; whether semantic or atlas hits hydrate exact evidence before use; and whether stale or misleading feedback affects review queues without erasing the underlying record.

## 10 Memory Authority and User Control

Memory systems can easily become intrusive. The more natural continuity feels, the more important it becomes to show where continuity comes from. A user should not have to wonder, "Why did the assistant know that?"

The architecture assigns authority explicitly.

The operator owns memory inclusion policy, deletion, privacy scope, review thresholds, and correction of stale or misleading records. The host owns stable IDs, provenance, access control, retrieval bounds, evidence hydration, reducer execution, and prompt injection. The model may propose summaries, relevance links, reducer actions, recall requests, and interpretations. Evidence determines what can honestly be claimed.

This split supports user operations such as:

- inspect stored memory;
- inspect why context was injected;
- mark a memory stale or misleading;
- disable a trigger;
- delete or archive a record according to policy;
- require confirmation before storing stable personal facts;
- export logs and receipts;
- distinguish inferred memory from user-explicit memory.

User control should not be cosmetic. A memory that cannot be inspected, corrected, scoped, or disabled is not simply convenient personalization. It is hidden state shaping the user's interaction.

## 11 Failure Modes

Evidence-bearing host memory is designed around common failure modes.

Summary replaces evidence. The system begins treating a compressed summary as historical truth. Rail: every substantive summary retains evidence refs and can descend to exact records.

Deep recall becomes generic RAG. The host dumps semantically nearby chunks into context. Rail: retrieval begins from a stated question or missing detail and stops when answered, unresolved, or budget-limited.

Summary drift. Repeated summarization changes what old events mean. Rail: summaries are versioned interpretations over stable evidence.

False memory completion. The model fills gaps from plausibility. Rail: recall states include exact recovered, partial, broad only, contradiction, no supporting record, source unavailable, and budget exhausted.

Stale personalization. Old preferences or project states remain active after they stop being true. Rail: records carry currentness, status, last-updated time, confidence, and feedback receipts.

Over-injection. Too many memories enter the prompt and distort the current turn. Rail: strict budgets, ranked candidates, user selection, and one-turn attachment boundaries.

Privacy leakage. A broad summary exposes restricted source material. Rail: scope inheritance and access checks from underlying evidence.

Cross-model misread. A new model interprets old buckets incorrectly. Rail: regenerate or migrate bucket maps deliberately, preserve old state, and log migration.

Memory theater. The interface shows a human approval step but users cannot meaningfully inspect what will be injected. Rail: preview exact payloads, show provenance, and make approval consequential.

## 12 Future Evaluation Agenda

The architecture can be evaluated empirically after the design contribution is stated. Useful studies would compare host-memory configurations across realistic assistant tasks.

A first evaluation could compare four host-memory conditions:

1. no persistent memory;
2. rolling summary only;
3. vector or semantic retrieval without user-selected evidence hydration;
4. evidence-bearing host memory with semantic routing, exact hydration, user selection, and receipts.

Tasks should include project continuation, technical review, personal preference handling, correction of stale facts, exact recall, and contradiction detection. Evaluation should not stop at user satisfaction or response fluency.

Suggested metrics include:

- exact recall accuracy;
- source attribution accuracy;
- stale-memory detection;
- user ability to explain why context appeared;
- reduction in irrelevant injected context;
- false-positive retrieval rate;
- correction persistence after user feedback;
- ability to distinguish summary from evidence;
- privacy-scope violations;
- token cost per useful recalled detail;
- task success after long gaps.

The evaluation should separate system-level and user-level outcomes:

| Evaluation layer    | Example measure                                                    | Why it matters                                |
|---------------------|--------------------------------------------------------------------|-----------------------------------------------|
| Retrieval exactness | Does the system recover the source event or only a nearby summary? | Tests whether memory remains evidence-bearing |

| Evaluation layer       | Example measure                                                                  | Why it matters                                          |
|------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------|
| Attribution            | Can the answer identify which record supported it?                               | Tests whether users can inspect memory origin           |
| Boundary preservation  | Is injected evidence cleared after one turn when required?                       | Tests whether recall becomes silent state               |
| Conflict handling      | Does the system surface conflicting records rather than merge them?              | Tests resistance to false memory completion             |
| Correction persistence | Does stale or misleading feedback affect later review without deleting evidence? | Tests governable memory revision                        |
| User comprehension     | Can users explain why the assistant remembered something?                        | Tests whether transparency is usable, not merely logged |

For implementation-level evaluation, the current assistant-memory stack already suggests concrete tests: proposed placement inertia, user approval, semantic recall, bounded hydration, selected attachment rendering, receipt creation, conflicting-memory status, not-found status, and feedback-driven review signals that do not mutate cold logs.

For a future empirical or system-evaluation article, a compact study could use scripted long-gap assistant tasks over a fixed local corpus:

1. Ask the assistant to resume a prior project decision after intervening unrelated work.
2. Ask for an exact detail that is present in cold evidence but absent or distorted in summary.
3. Ask for a stale preference or superseded project status.
4. Ask for a query with two conflicting historical records.
5. Ask the user to explain why a memory appeared and whether it should shape the next turn.

Each condition can be scored for exactness, attribution, user comprehension, correction persistence, false recall, and context cost. The expected contribution is not that evidence-bearing host memory maximizes conversational charm. It is that the user can tell where continuity came from and can intervene before memory becomes hidden authority.

The strongest evidence for the design would be not that the assistant sounds more continuous, but that continuity becomes more auditable, correctable, and scope-faithful.

## 13 What This Paper Is Not Claiming

This paper does not claim to solve LLM memory generally. It does not provide infinite context, human memory, consciousness, or guaranteed truth. It does not claim that summaries, embeddings, or RAG are bad. It does not claim that every recall should require user approval. It does not claim that local-first systems are the only legitimate implementation path.

The narrower claim is that memory-like behavior in LLM assistants should be designed as evidence-bearing host infrastructure. Compressed memory is useful, but it should remain linked to exact evidence. The model can help shape and use memory, but the host should own persistence and injection. Users should be able to inspect why memory appears and correct it when it is wrong.

## 14 Conclusion

LLM memory should not be treated as a mysterious property of the model. It is an architecture of records, summaries, retrieval paths, injections, policies, and receipts. When those layers are collapsed, systems

become harder to audit and easier to anthropomorphize. When they are separated, memory becomes a governable part of the assistant stack.

The evidence-bearing host-memory model separates active context, rolling summary, semantic routing, and cold evidence. It lets broad shape guide retrieval without replacing exact history. It lets the model propose memory actions without owning persistence. It lets users select and inspect recalled evidence before it shapes the next turn. It records receipts so memory behavior can itself be reviewed.

The result is not perfect recall. It is something more modest and more useful: bounded continuity that can explain itself.

## REFERENCES

- ACM. (2026). \*Open access at ACM\*. <https://authors.acm.org/open-access>
- ACM Transactions on Interactive Intelligent Systems. (2026). \*Author guidelines\*. <https://dl.acm.org/journal/tiis/author-guidelines>
- Amershi, S., Weld, D., Vorvoreanu, M., Fournay, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., and Horvitz, E. (2019). Guidelines for human-AI interaction. \*Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems\*, Paper 3, 1-13. <https://doi.org/10.1145/3290605.3300233>
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. \*arXiv preprint\* arXiv:2312.10997. <https://arxiv.org/abs/2312.10997>
- Kulesza, T., Burnett, M., Wong, W.-K., and Stumpf, S. (2015). Principles of explanatory debugging to personalize interactive machine learning. \*Proceedings of the 20th International Conference on Intelligent User Interfaces\*, 126-137. <https://doi.org/10.1145/2678025.2701399>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. \*Advances in Neural Information Processing Systems\*, 33\*, 9459-9474. <https://arxiv.org/abs/2005.11401>
- Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. \*Artificial Intelligence\*, 267\*, 1-38. <https://doi.org/10.1016/j.artint.2018.07.007>
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. \*arXiv preprint\* arXiv:2310.08560. <https://arxiv.org/abs/2310.08560>
- Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. \*Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology\*, Article 2, 1-22. <https://doi.org/10.1145/3586183.3606763>
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. \*arXiv preprint\* arXiv:2303.11366. <https://arxiv.org/abs/2303.11366>